

SUPPLEMENTARY PROBLEM: PARTITIONING AND AVERAGE CASE ANALYSIS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem 1

You have an array A of integers. Develop an efficient algorithm to find the K th smallest element.

Example

Input:

- $A = [3, 1, -1, 5, 6, 4, 10, 11]$
- $K = 2$

Output:

- 1

Problem 1 Solution

PLEASE NOTE THAT YOU DO NOT NEED TO PROVIDE PSUEDOCODE, RUN-TIME ANALYSIS, OR PROOF FOR PARTITION - WE HAVE INCLUDED IT TO MAKE THE SOLUTION MORE COMPLETE.

Pseudocode

Algorithm 1 Find the K th Smallest Element in Array

Require: A is an array of n integers, $1 \leq K \leq n$

```
1: procedure QuickSelect( $A, 0, n - 1, K - 1$ )
2:   return rec_qs( $A, 0, n - 1, K - 1$ )
3: end procedure
4:
5: procedure rec_qs( $A, left, right, K$ )
6:   if  $left = right$  then
7:     return  $A[left]$ 
8:   end if
9:    $pivotIndex \leftarrow$  partition( $A, left, right$ )
10:  if  $K = pivotIndex$  then
11:    return  $A[K]$ 
12:  else if  $K < pivotIndex$  then
13:    return rec_qs( $A, left, pivotIndex - 1, K$ )
14:  else
15:    return rec_qs( $A, pivotIndex + 1, right, K$ )
16:  end if
17: end procedure
18:
19: procedure partition( $A, left, right$ )
20:    $pivot \leftarrow A[right]$ 
21:    $i \leftarrow left$ 
22:    $j \leftarrow left$ 
23:   while  $j < right$  do
24:     if  $A[j] < pivot$  then
25:       swap  $A[i]$  and  $A[j]$ 
26:        $i \leftarrow i + 1$ 
27:     end if
28:      $j \leftarrow j + 1$ 
29:   end while
30:   swap  $A[i]$  and  $A[right]$ 
31:   return  $i$ 
32: end procedure
```

Proof of Correctness

Proof: partition

Loop Invariant:

1. $A[left..i - 1] < pivot$
2. $A[i..j - 1] \geq pivot$
3. $A[right] = pivot$

Proof:

- **Initialization** Before the loop begins, $i = left$ and $j = left$. The subarray $A[left..left - 1]$ doesn't exist, so (1) and (2) hold vacuously. Finally, we have initialized the pivot to be $A[right]$, so (3) holds.
- **Maintenance:** Assume the L.I. holds true at some arbitrary iteration. We must consider two cases:
 1. $A[j] < pivot$
 - We now know the elements $A[left..i - 1]$ and $A[j]$ are less than pivot.
 - We swap $A[i]$ and $A[j]$.
 - After the swap, $A[left..i] < pivot$.
 - Finally, i is incremented, so the subarray $A[left..i - 1] < pivot$.
 - $A[i \dots j - 1]$ continues to satisfy that all elements are greater than or equal to $pivot$ because only i in this case. Thus, if $A[i] \dots A[j - 1]$ were already greater than or equal to the pivot, swapping $A[i]$ and $A[j]$ maintains the invariant by moving $A[j]$ (which is less than the pivot) to $A[i]$ and incrementing i .
 2. $A[j] \geq pivot$
 - In this case, we do not swap $A[j]$ with $A[i]$
 - This maintains $A[i..j - 1]$ as all elements greater than or equal to $pivot$ since $A[j] \geq pivot$.
 - Additionally, $A[left..i - 1]$ remains less than the pivot, as we have not changed any elements in this range.

Finally, we increment j to consider the next element in the array. We have not changed the value of $pivot$, thus $pivot$ still holds $A[right]$ Therefore, the loop invariant conditions hold at the end of each iteration.

- **Termination:** At termination, $j = right$. All elements in the range $A[left..right - 1]$ have been compared with the pivot. (1) and (2) are still satisfied). Finally, we swap $A[i]$ with $A[right]$, placing the pivot in its correct position i .

Now, let's move onto proving that `rec_qs` works correctly. The goal is to show that $rec_qs(A, left, right, K)$ correctly returns the K th smallest element of the array A .

Base Case

When $left = right$, that means the subarray we are considering is of length 1. The algorithm directly returns $A[left]$. This element is trivially the smallest (and only) element in the array, and it is correct to return it as the K th smallest element.

Inductive Hypothesis

Assume that for any array of size m , where $1 \leq m < n$, `rec_qs` correctly returns the K th smallest element.

Inductive Step

The partition function is called, which rearranges the array and returns an index $pivotIndex$ such that:

- All elements in $A[left..pivotIndex - 1] < A[pivotIndex]$.
- All elements in $A[pivotIndex + 1..right] \geq A[pivotIndex]$.
- Since partition correctly places the pivot element $A[pivotIndex]$ in its sorted position, we know that $A[pivotIndex]$ is the $pivotIndex - left + 1$ th smallest element of the array $A[left..right]$.

We then have to consider the following cases:

1. $K = pivotIndex$

- In this case, $A[pivotIndex]$ is exactly the K th smallest element because partition ensures that all elements to the left are smaller and all elements to the right are larger or equal.
- Since $A[pivotIndex]$ is the K th smallest element, the algorithm correctly returns $A[K]$.

2. $K < pivotIndex$

- Here, the K th smallest element must be in the left partition $A[left..pivotIndex - 1]$ due to the properties from `partition`, which has fewer than n . We reduce the problem to finding the K th smallest element in the left subarray. Since this subarray has fewer elements than the original array, by the inductive hypothesis, `rec_qs` correctly finds the K th smallest element in this smaller subarray.

3. $K > pivotIndex$

- Here, the K th smallest element must be in the right partition $A[pivotIndex + 1..right]$ due to the properties from `partition`, which has fewer than n . We reduce the problem to finding the K th smallest element in the right subarray. Since this subarray has fewer elements than the original array, by the inductive hypothesis, `rec_qs` correctly finds the K th smallest element in this smaller subarray.

Runtime Analysis

partition iterates over the elements from left to right, making one pass through the subarray and performing comparisons and swaps. Thus, the runtime for a single call to **partition** is $\Theta(n)$

Average Case:

In this case, the **partition** divides the array in approximately half, meaning the recurrence is

$$T(n) = T(n/2) + \Theta(n)$$

First, calculate the divide-term runtime. Since $a = 1$ and $b = 2$, we have:

$$n^{\log_b(a)} = n^{\log_2(1)} = n^0 = 1$$

Notice that the divide-term is polynomially smaller than the combine term, i.e. $n^0 = O(n^{1-\epsilon})$ for some $\epsilon > 0$. Therefore, Case 3 of Master Method will apply here, but first we must check that the regularity condition holds.

$$af\left(\frac{n}{b}\right) \leq cf(n)$$

for some constant $c < 1$. Since $f(n) = \Theta(n)$, $a = 1$, and $b = 2$, then

$$f(n/2) = n/2 \leq cn$$

holds when $c = 2/3$. Therefore, regularity holds and Case 3 Master Method applies, meaning that the runtime of the algorithm is

$$T(n) = O(n)$$

Worst-case runtime:

The recurrence relation for the worst-case scenario is:

$$T(n) = T(n-1) + O(n)$$

Expanding this recurrence gives:

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &\vdots \\ &= T(0) + \sum_{i=1}^n i \end{aligned}$$

The sum $\sum_{i=1}^n i$ is a the following arithmetic series:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = O(n^2)$$

Thus, in the worst case, the time complexity of our algorithm is:

$$T(n) = O(n^2)$$

Problem 2

You work as a security agent at the airport. As part of your job, you are required to call your manager whenever you saw a person who is taller than all other people you saw so far on that day. Assuming that people pass the security in random order, provide a tight asymptotic bound on the expected number of calls you will make if you saw n people in total.

Problem 2 Solution

Since people are passing the security in random order, our goal is to find the expected number calls. Let X represent the expected number of calls then:

$$E[X] = \sum_{c=1}^n \Pr\{X = c\}c$$

This probability is difficult to compute. However, we can use the indicator random variable to indicate if the current person c_i is the tallest.

Let C_i denote the indicator random variable which equals 1 if person c_i is the tallest, 0 otherwise. The sum of these indicator random variables, X , will provide us the total number of calls. That is,

$$C_i = I\{\text{person } c_i \text{ is tallest so far}\}$$
$$X = \sum_{i=1}^n C_i$$

Using the linearity of expectation (probability jargon) allows us to switch the order of the sum and the expectation which makes the problem a lot easier.

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^n C_i\right] = \sum_{i=1}^n E[C_i] \\ &= \sum_{i=1}^n [\Pr\{c_i \text{ is tallest}\}1 + \Pr\{c_i \text{ is not tallest}\}0] \\ &= \sum_{i=1}^n \Pr\{c_i \text{ is tallest}\} \end{aligned}$$

All we need to do is determine what the probability is of a person being taller than all the people in line before them. Since people arrive in random order, then they all have an equal chance of

best the tallest. Let's look at the individual probabilities for each person c_i .

$$\begin{aligned}
 \Pr\{c_1 \text{ is tallest}\} &= 1 \\
 \Pr\{c_2 \text{ is tallest}\} &= 1/2 \\
 \Pr\{c_3 \text{ is tallest}\} &= 1/3 \\
 &\vdots \\
 \Pr\{c_i \text{ is tallest}\} &= 1/i \\
 &\vdots \\
 \Pr\{c_n \text{ is tallest}\} &= 1/n
 \end{aligned}$$

To start, the first person has a 100% chance of being the tallest so far since no people arrived before them. Let's say at some arbitrary point that $i - 1$ people have arrived so far. Since people arrive in random order, the i -th person will then have an equally like chance as the first $i - 1$ people of being the tallest so far. Therefore, the i -th person has a $1/i$ probability of being the tallest at that moment.

If we plug in these probabilities into the sum we created earlier, we see that the expected number of calls is

$$X = \sum_{i=1}^n \Pr\{c_i \text{ is tallest}\} = \sum_{i=1}^n \frac{1}{i} \approx \ln n$$

Therefore, the expected number of calls you will make if you saw n people in total is asymptotically bounded by $\Theta(\log n)$